

Auth, DevOps & Security – Complete Guide

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 19 Oct 2026 – 25 Oct 2026 (Days 64–70)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm** & **9:00–10:00pm** · Sat–Sun **9:00am–12:00pm**, **3:00–5:30pm**, **6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import `calendar/mern-mastery-study.ics`

JWT, OAuth, sessions, Docker, AWS, testing, CI/CD, and interview preparation for MERN developers.

Table of Contents

1. Authentication vs Authorization
2. JWT Deep Dive
3. OAuth 2.0 & OpenID Connect
4. Sessions & Cookies
5. Password Security (bcrypt)
6. Docker Fundamentals
7. Docker Compose
8. AWS Basics for Developers
9. Testing: Jest, Supertest, RTL
10. CI/CD with GitHub Actions
11. Security Best Practices
12. Interview Q&A
13. Practice Links & Exercises

1. Authentication vs Authorization

Concept	Question	Example
Authentication (AuthN)	Who are you?	Login with email/password
Authorization (AuthZ)	What can you do?	Admin vs user role permissions

Request → AuthN (verify identity) → AuthZ (check permissions) → Handler

Common mistakes:

- Checking role in frontend only (must enforce on server)
- Confusing 401 (Unauthorized = not authenticated) with 403 (Forbidden = authenticated but denied)

2. JWT Deep Dive

Structure

```
Header.Payload.Signature  
  
eyJhbGciOiJIUzI1NiIs... ← Base64URL(Header)  
.  
eyJ1c2VySWQiOiJEsInJvbGUi... ← Base64URL(Payload)  
.  
SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c ← Signature
```

Header: { "alg": "HS256", "typ": "JWT" } **Payload:** Claims — sub, iat, exp, custom (userId, role) **Signature:** HMACSHA256(base64(header) + "." + base64(payload), secret)

Access + Refresh Token Flow

1. POST /login { email, password }
2. Server validates → issues accessToken (15m) + refreshToken (7d)
3. Access token → Authorization: Bearer <token> (memory or short-lived cookie)
4. Refresh token → httpOnly, Secure, SameSite=Strict cookie
5. On 401 → POST /refresh with refresh cookie → new access token
6. On logout → invalidate refresh token server-side (DB/redis denylist)

TypeScript Implementation

```
import jwt from "jsonwebtoken";
import { Request, Response, NextFunction } from "express";

interface TokenPayload {
  userId: string;
  role: "user" | "admin";
}

export function signAccessToken(payload: TokenPayload): string {
  return jwt.sign(payload, process.env.JWT_ACCESS_SECRET!, {
    expiresIn: "15m",
    issuer: "myapp",
    audience: "myapp-users",
  });
}

export function signRefreshToken(userId: string): string {
  return jwt.sign({ userId }, process.env.JWT_REFRESH_SECRET!, {
    expiresIn: "7d",
  });
}

export function authMiddleware(req: Request, res: Response, next: NextFunction) {
  const header = req.headers.authorization;
  if (!header?.startsWith("Bearer ")) {
    return res.status(401).json({ error: "Missing token" });
  }
  try {
    const token = header.slice(7);
    req.user = jwt.verify(token, process.env.JWT_ACCESS_SECRET!) as TokenPayload;
    next();
  } catch {
    return res.status(401).json({ error: "Invalid or expired token" });
  }
}

export function requireRole(...roles: string[]) {
  return (req: Request, res: Response, next: NextFunction) => {
    if (!req.user || !roles.includes(req.user.role)) {
      return res.status(403).json({ error: "Forbidden" });
    }
    next();
  };
}
```

JWT Security Rules

Rule	Why
Short access token expiry	Limits damage if stolen
Refresh token rotation	Detect token reuse attacks
httpOnly cookies for refresh	Blocks JavaScript access (XSS)
Never store JWT in localStorage	XSS can exfiltrate
Use RS256 for microservices	Private key signs, public key verifies
Validate <code>iss</code> , <code>aud</code> , <code>exp</code>	Prevent token misuse

3. OAuth 2.0 & OpenID Connect

OAuth 2.0 Roles

- **Resource Owner:** User
- **Client:** Your app
- **Authorization Server:** Google, GitHub
- **Resource Server:** API holding user data

Authorization Code Flow (Most Secure)

```
User → Click "Login with Google"
      → Redirect: GET accounts.google.com/o/oauth2/auth?client_id=...&redirect_uri=...&
      → User consents
      → Redirect: GET yourapp.com/callback?code=AUTH_CODE
      → Server: POST token endpoint with code + client_secret
      → Receive: access_token, id_token (OIDC), refresh_token
      → Fetch user profile OR decode id_token
      → Create/link user in DB → issue YOUR app's JWT/session
```

Passport.js Example

```
import passport from "passport";
import { Strategy as GoogleStrategy } from "passport-google-oauth20";

passport.use(new GoogleStrategy({
  clientId: process.env.GOOGLE_CLIENT_ID!,
  clientSecret: process.env.GOOGLE_CLIENT_SECRET!,
  callbackURL: "/auth/google/callback",
}, async (accessToken, refreshToken, profile, done) => {
  const user = await findOrCreateUser({
    provider: "google",
    providerId: profile.id,
    email: profile.emails?.[0]?.value,
    name: profile.displayName,
  });
  done(null, user);
}));
```

Next.js: NextAuth.js

```
// app/api/auth/[...nextauth]/route.ts
import NextAuth from "next-auth";
import GoogleProvider from "next-auth/providers/google";
import CredentialsProvider from "next-auth/providers/credentials";

export const authOptions = {
  providers: [
    GoogleProvider({
      clientId: process.env.GOOGLE_ID!,
      clientSecret: process.env.GOOGLE_SECRET!,
    }),
    CredentialsProvider({
      name: "Credentials",
      credentials: { email: {}, password: {} },
      async authorize(credentials) {
        const user = await validateUser(credentials);
        return user ?? null;
      },
    }),
  ],
  session: { strategy: "jwt" as const },
  callbacks: {
    async jwt({ token, user }) {
      if (user) token.role = user.role;
      return token;
    },
  },
};
```

4. Sessions & Cookies

Session-Based Auth

Login → Server creates session ID → Store in Redis/DB
→ Set cookie: sessionId=abc123; HttpOnly; Secure; SameSite=Strict
→ Each request: lookup session → attach user to req

```
import session from "express-session";
import RedisStore from "connect-redis";
import { createClient } from "redis";

const redisClient = createClient({ url: process.env.REDIS_URL });
await redisClient.connect();

app.use(session({
  store: new RedisStore({ client: redisClient }),
  secret: process.env.SESSION_SECRET!,
  resave: false,
  saveUninitialized: false,
  cookie: {
    httpOnly: true,
    secure: process.env.NODE_ENV === "production",
    sameSite: "strict",
    maxAge: 24 * 60 * 60 * 1000,
  },
}));
```

JWT vs Sessions

Aspect	JWT	Sessions
State	Stateless (mostly)	Stateful (server store)
Revocation	Hard (need denylist)	Easy (delete session)
Scale	Easy horizontal	Needs shared store (Redis)
Mobile/SPA	Excellent	Cookie limitations
Size	Larger headers	Small cookie (session ID)

5. Password Security (bcrypt)

```
import bcrypt from "bcrypt";

const SALT_ROUNDS = 12; // 10-12 typical; higher = slower brute force

export async function hashPassword(plain: string): Promise<string> {
  return bcrypt.hash(plain, SALT_ROUNDS);
}

export async function verifyPassword(plain: string, hash: string): Promise<boolean> {
  return bcrypt.compare(plain, hash);
}

// Registration
const passwordHash = await hashPassword(req.body.password);
await User.create({ email, passwordHash }); // NEVER store plain text

// Login – constant-time compare via bcrypt
const user = await User.findOne({ email });
if (!user || !(await verifyPassword(password, user.passwordHash))) {
  return res.status(401).json({ error: "Invalid credentials" });
}
```

Additional measures:

- Minimum password length (12+), check against breached passwords (HaveIBeenPwned API)
- Rate limit login attempts (5/min per IP)
- Account lockout after N failures
- MFA for sensitive apps

6. Docker Fundamentals

Core Concepts

Term	Meaning
Image	Read-only template (layers)
Container	Running instance of image
Dockerfile	Recipe to build image
Volume	Persistent storage
Network	Container communication

Dockerfile for Node.js MERN App

```
# Multi-stage build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine AS production
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --only=production && npm cache clean --force
COPY --from=builder /app/dist ./dist
USER node
EXPOSE 3000
HEALTHCHECK --interval=30s CMD wget -qO- http://localhost:3000/health || exit 1
CMD ["node", "dist/server.js"]
```

Essential Commands

```
docker build -t myapp:1.0 .
docker run -d -p 3000:3000 --name myapp -e NODE_ENV=production myapp:1.0
docker ps                                # running containers
docker logs -f myapp
docker exec -it myapp sh
docker stop myapp && docker rm myapp
docker images
docker rmi myapp:1.0
docker system prune -a                  # cleanup unused
```

.dockerignore

```
node_modules
.git
.env
*.md
coverage
.next
dist
```

7. Docker Compose

```
# docker-compose.yml
version: "3.9"

services:
  app:
    build: .
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
      - MONGODB_URI=mongodb://mongo:27017/myapp
      - REDIS_URL=redis://redis:6379
    depends_on:
      mongo:
        condition: service_healthy
      redis:
        condition: service_started
    restart: unless-stopped

  mongo:
    image: mongo:7
    volumes:
      - mongo_data:/data/db
    healthcheck:
      test: echo 'db.runCommand("ping").ok' | mongosh localhost:27017/test --quiet
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    volumes:
      - redis_data:/data

  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - app

volumes:
  mongo_data:
  redis_data:
```

```
docker compose up -d          # start all services
docker compose down          # stop and remove
docker compose logs -f app
docker compose exec app sh
docker compose up --build     # rebuild images
```

8. AWS Basics for Developers

Core Services

Service	Purpose	MERN Use Case
EC2	Virtual servers	Host Node.js app
ECS/EKS	Containers	Docker deployments
S3	Object storage	Images, uploads, static assets
RDS	Managed SQL	PostgreSQL if not MongoDB
DocumentDB	MongoDB-compatible	Managed Mongo
ElastiCache	Redis/Memcached	Sessions, caching
CloudFront	CDN	Static asset delivery
Route 53	DNS	Domain routing
ALB	Load balancer	Traffic distribution
IAM	Access control	Roles, policies
Secrets Manager	Secrets	API keys, DB passwords
CloudWatch	Monitoring	Logs, metrics, alarms
Lambda	Serverless functions	Webhooks, cron jobs
SQS/SNS	Messaging	Async jobs, notifications

Typical MERN Deployment Architecture

```
Route 53 → CloudFront (static) → S3
          → ALB → ECS (Node API) → DocumentDB
                                → ElastiCache (Redis)
                                → S3 (uploads)
CloudWatch ← logs/metrics from all services
```

IAM Best Practices

- Never use root account for daily work
- Principle of least privilege
- Use IAM roles for EC2/ECS (not access keys on disk)
- Enable MFA on root and admin accounts

S3 Upload (Presigned URL Pattern)

```
import { S3Client, PutObjectCommand } from "@aws-sdk/client-s3";
import { getSignedUrl } from "@aws-sdk/s3-request-presigner";

const s3 = new S3Client({ region: "ap-south-1" });

export async function getUploadUrl(key: string, contentType: string) {
  const command = new PutObjectCommand({
    Bucket: process.env.S3_BUCKET!,
    Key: key,
    ContentType: contentType,
  });
  return getSignedUrl(s3, command, { expiresIn: 300 });
}
```

9. Testing: Jest, Supertest, RTL

Jest Unit Tests

```
// __tests__/auth.service.test.ts
import { hashPassword, verifyPassword } from "../services/auth";

describe("Auth Service", () => {
  it("hashes and verifies password", async () => {
    const hash = await hashPassword("secret123");
    expect(hash).not.toBe("secret123");
    expect(await verifyPassword("secret123", hash)).toBe(true);
    expect(await verifyPassword("wrong", hash)).toBe(false);
  });
});
```

Supertest Integration Tests

```
// __tests__/auth.routes.test.ts
import request from "supertest";
import app from "../app";
import { connectDB, disconnectDB } from "../db";

beforeAll(async () => { await connectDB(); });
afterAll(async () => { await disconnectDB(); });

describe("POST /api/auth/login", () => {
  it("returns 401 for invalid credentials", async () => {
    const res = await request(app)
      .post("/api/auth/login")
      .send({ email: "bad@test.com", password: "wrong" });
    expect(res.status).toBe(401);
  });

  it("returns tokens for valid login", async () => {
    const res = await request(app)
      .post("/api/auth/login")
      .send({ email: "user@test.com", password: "password123" });
    expect(res.status).toBe(200);
    expect(res.body).toHaveProperty("accessToken");
  });
});
```

React Testing Library

```
// LoginForm.test.tsx
import { render, screen, waitFor } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import { LoginForm } from "../LoginForm";

test("shows error on failed login", async () => {
  const user = userEvent.setup();
  render(<LoginForm />);
  await user.type(screen.getByLabelText(/email/i), "bad@test.com");
  await user.type(screen.getByLabelText(/password/i), "wrong");
  await user.click(screen.getByRole("button", { name: /login/i }));
  await waitFor(() => {
    expect(screen.getByText(/invalid credentials/i)).toBeInTheDocument();
  });
});
```

Testing Pyramid

```

      /\
     /E2E\      Playwright/Cypress – few, critical paths
    /-----\
   /Integration\ Supertest – API routes, DB
  /-----\
 /   Unit Tests   \  Jest – pure functions, utils
/-----\

```

10. CI/CD with GitHub Actions

```
# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      mongo:
        image: mongo:7
        ports: ["27017:27017"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: "npm"
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck
      - run: npm test -- --coverage
      - uses: codecov/codecov-action@v4
        if: always()

  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/build-push-action@v5
        with:
          context: .
          push: false
          tags: myapp:${{ github.sha }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

  deploy:
    needs: build
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
```

```
- name: Deploy to production  
  run: echo "Deploy via ECS/Railway/Vercel"
```

CI/CD Best Practices

- Run lint, typecheck, tests on every PR
- Cache dependencies (`actions/cache` , npm cache)
- Use environment secrets for deploy keys
- Separate staging and production pipelines
- Require PR reviews + passing CI before merge

11. Security Best Practices

OWASP Top 10 (Developer Focus)

1. **Broken Access Control** — enforce AuthZ server-side
2. **Cryptographic Failures** — HTTPS, bcrypt, secure secrets
3. **Injection** — parameterized queries, validate input
4. **Insecure Design** — threat modeling early
5. **Security Misconfiguration** — disable debug in prod, secure headers
6. **Vulnerable Components** — `npm audit` , Dependabot
7. **Auth Failures** — MFA, rate limiting, secure sessions
8. **Integrity Failures** — verify CI/CD pipeline integrity
9. **Logging Failures** — log auth events, no secrets in logs
10. **SSRF** — validate URLs, block internal network access

Security Headers (Helmet.js)

```
import helmet from "helmet";
app.use(helmet());
app.use(helmet.contentSecurityPolicy({
  directives: {
    defaultSrc: ['self'],
    scriptSrc: ['self'],
    styleSrc: ['self', 'unsafe-inline'],
  },
}));
```

Rate Limiting

```
import rateLimit from "express-rate-limit";

const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  message: { error: "Too many login attempts" },
});
app.post("/api/auth/login", loginLimiter, loginHandler);
```

Input Validation (Zod)

```
import { z } from "zod";

const registerSchema = z.object({
  email: z.string().email().max(255),
  password: z.string().min(12).max(128),
  name: z.string().min(1).max(100),
});
```

12. Interview Q&A

Q: JWT vs session — when to use which? A: JWT for stateless APIs, mobile, microservices. Sessions when you need easy revocation and server-side control.

Q: Where to store JWT on client? A: Access token in memory; refresh in httpOnly Secure cookie. Avoid localStorage.

Q: How does OAuth differ from JWT? A: OAuth is an authorization framework (delegated access). JWT is a token format. OAuth often issues JWTs as id/access tokens.

Q: Explain refresh token rotation. A: Each refresh use issues new refresh token and invalidates old one. Detects reuse (stolen token) and revokes family.

Q: What is bcrypt salt rounds? A: Cost factor — 2^n rounds iterations. Higher = slower hashing, harder brute force. 10-12 typical.

Q: Docker multi-stage build benefit? A: Smaller production image — build tools stay in builder stage, only runtime artifacts copied.

Q: Difference between CMD and ENTRYPOINT? A: ENTRYPOINT defines executable; CMD provides default args. Together they define container startup.

Q: How do you secure secrets in CI/CD? A: GitHub Secrets, AWS Secrets Manager, never commit .env, rotate regularly.

Q: What is SameSite cookie attribute? A: Controls cross-site cookie sending. Strict prevents CSRF; Lax allows top-level navigation.

Q: How to test authenticated routes with Supertest? A: Login first, extract token/cookie, pass in subsequent requests via `.set('Authorization', 'Bearer ...')`.

13. Practice Links & Exercises

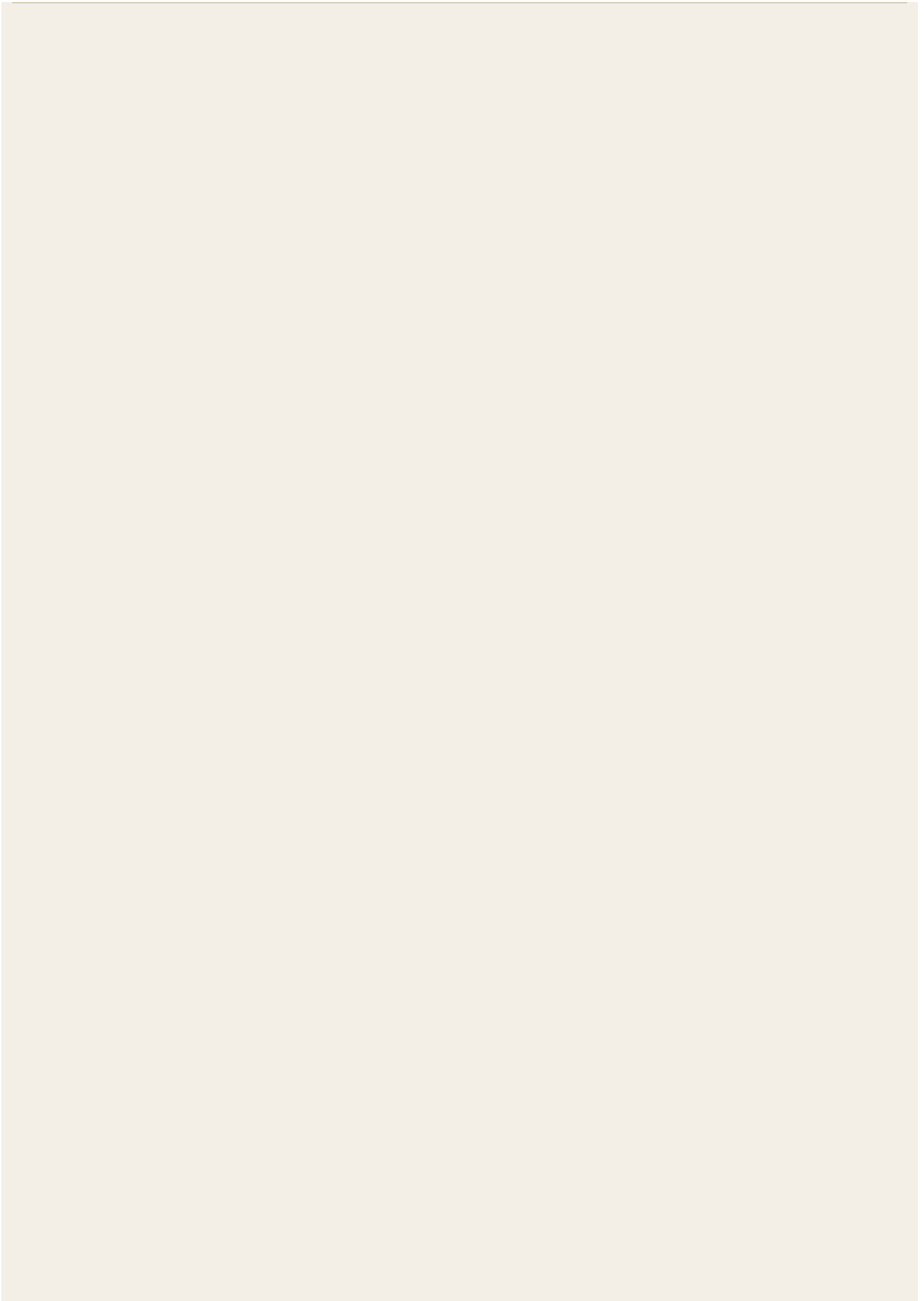
Resource	URL	Topic
JWT.io Debugger	jwt.io	Decode/verify tokens
OAuth Playground	developers.google.com/oauthplayground	OAuth flows
Docker Docs	docs.docker.com/get-started	Containers
Play with Docker	labs.play-with-docker.com	Free sandbox
AWS Skill Builder	skillbuilder.aws	AWS fundamentals
OWASP Cheat Sheets	cheatsheetseries.owasp.org	Security
Testing Library Docs	testing-library.com	RTL
GitHub Actions Docs	docs.github.com/actions	CI/CD

Hands-On Exercises

1. Build JWT auth with access/refresh rotation and httpOnly cookies
2. Add Google OAuth with Passport or NextAuth
3. Dockerize MERN app with Mongo + Redis via compose
4. Write Supertest suite for all auth endpoints
5. Create GitHub Actions pipeline: lint → test → build Docker image
6. Deploy to AWS ECS or Railway with environment secrets
7. Add Helmet, rate limiting, Zod validation to Express API
8. Implement presigned S3 upload from React frontend

Quick Reference

```
AuthN: bcrypt + JWT/session + OAuth
AuthZ: RBAC middleware, server-side checks
DevOps: Dockerfile → compose → CI → deploy
Test: Jest (unit) → Supertest (API) → RTL (UI)
Security: HTTPS, httpOnly, rate limit, validate, audit
```



Product Company Interview Bank – Authentication and DevOps (50 Questions)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Target companies: Google, Meta, Microsoft, Amazon, Adobe, Atlassian, Uber, Flipkart **Rule:** Full terms only — spell out JWT as JSON Web Token, API as Application Programming Interface, etc.

Question 1: What is authentication?

Answer: Authentication is the process of verifying who a user is — proving identity. Common methods: username and password, email magic link, biometrics, or social login (Google, GitHub). When you log in successfully, the system confirms "this is really Harsh" before granting access. Authentication answers the question: "Are you who you claim to be?" Failed authentication returns 401 Unauthorized.

Question 2: What is authorization?

Answer: Authorization happens after authentication. It determines what an authenticated user is allowed to do — which pages they can view, which data they can edit, which actions they can perform. Example: Harsh is authenticated (logged in) but not authorized to delete another user's post (403 Forbidden). Role-Based Access Control (RBAC) is a common authorization model with roles like admin, editor, and viewer.

Question 3: What is the difference between authentication and authorization?

Answer: Authentication = identity verification (login). Authorization = permission check (access control). You must authenticate first, then authorize. Analogy: authentication is showing your ID at a building entrance; authorization is whether your badge lets you enter the server room. In code: middleware checks JWT (authentication), then checks `user.role === 'admin'` (authorization).

Question 4: What is a JSON Web Token (JWT)?

Answer: A JSON Web Token (JWT) is a compact, URL-safe string that carries claims (data) about a user, signed by the server so it cannot be tampered with. Structure: Header.Payload.Signature (three Base64 parts separated by dots). The payload might contain `{ userId: "123", role: "user", exp: 1700000000 }`. Clients send JWTs in the Authorization header. The server verifies the signature without querying a database — stateless authentication.

Question 5: What are the parts of a JSON Web Token (JWT)?

Answer: **Header:** algorithm (HS256, RS256) and token type. **Payload:** claims — registered (exp, iat), public, and private (userId, email, role). **Signature:** created by signing header + payload with a secret (symmetric) or private key (asymmetric). The signature ensures if anyone changes the payload, verification fails. Never put sensitive data (passwords) in the payload — it is Base64-encoded, not encrypted.

Question 6: What is the difference between symmetric and asymmetric JWT signing?

Answer: Symmetric (HS256): same secret signs and verifies — simple, used when one server handles both. Asymmetric (RS256): private key signs, public key verifies — safer for microservices where multiple services verify tokens but only the auth service holds the private key. Asymmetric is common at large companies (Google, Auth0). Symmetric is fine for monolithic MERN apps with one backend.

Question 7: What is a refresh token vs an access token?

Answer: An access token is short-lived (15 minutes to 1 hour) and sent with every Application Programming Interface (API) request — if stolen, damage is limited. A refresh token is long-lived (days/weeks), stored securely (HTTP-only cookie), and used only to get new access tokens when the access token expires. This balances security (short exposure) with user experience (stay logged in without re-entering password).

Question 8: Where should you store tokens on the frontend?

Answer: Best practice: Access token in memory (JavaScript variable) or short-lived; refresh token in HTTP-only, Secure, SameSite cookie — not accessible to JavaScript, protecting against Cross-Site Scripting (XSS). **Avoid:** localStorage for tokens — any XSS script can steal them. localStorage is acceptable only for non-sensitive preferences. For Single Page Applications, the HTTP-only cookie + refresh flow is industry standard.

Question 9: What is OAuth 2.0?

Answer: OAuth 2.0 is an authorization framework that lets users grant third-party apps limited access without sharing passwords. Example: "Log in with Google" — your app gets an access token from Google to read basic profile info, but never sees the user's Google password. OAuth handles delegated authorization. OpenID Connect (OIDC) builds on OAuth 2.0 to add authentication (identity layer with an ID token).

Question 10: What is OpenID Connect (OIDC)?

Answer: OpenID Connect is an identity layer on top of OAuth 2.0. While OAuth answers "can this app access my data?", OIDC answers "who is this user?" It adds an ID token (a JWT with user identity claims like name, email, sub/user ID). "Sign in with Google" typically uses OIDC — you get both authentication (ID token) and authorization (access token for Google APIs if needed).

Question 11: Explain the OAuth 2.0 Authorization Code flow.

Answer: (1) User clicks "Login with Google." (2) App redirects to Google's authorization server. (3) User consents. (4) Google redirects back with an authorization code. (5) Your backend exchanges the code for tokens (access + refresh + ID token) using client secret — secret stays on server, never in browser. (6) Backend creates a session or JWT for the user. This is the most secure OAuth flow for web apps.

Question 12: What is Cross-Site Request Forgery (CSRF) and how do you prevent it?

Answer: CSRF tricks a logged-in user's browser into making unwanted requests to your site (e.g., transfer money). Prevention: SameSite cookies (Strict or Lax), CSRF tokens in forms (server generates token, client sends it back), checking Origin/Referer headers. JSON Web Token (JWT) in Authorization header (not cookies) is naturally CSRF-resistant because browsers do not auto-send custom headers on cross-site requests.

Question 13: What is Cross-Site Scripting (XSS)?

Answer: XSS injects malicious JavaScript into your site that runs in other users' browsers — stealing cookies, tokens, or performing actions as the victim. Prevention: escape user input before rendering, use Content Security Policy headers, avoid `dangerouslySetInnerHTML`, sanitize HTML with libraries like DOMPurify, use HTTP-only cookies. Stored XSS persists in database; reflected XSS comes from URL parameters.

Question 14: What are HTTP-only, Secure, and SameSite cookie flags?

Answer: HttpOnly: JavaScript cannot read the cookie — protects against XSS token theft.

Secure: Cookie sent only over HTTPS — prevents interception on insecure networks.

SameSite: Controls cross-site sending — `Strict` blocks all cross-site; `Lax` allows top-level

navigation; **None** requires Secure. Set all three on session and refresh token cookies in production.

Question 15: What is password hashing and why not encryption?

Answer: Hashing is one-way — you cannot recover the original password from the hash. Encryption is two-way — you can decrypt with a key. Passwords should be hashed (bcrypt, argon2) because the server never needs the original password again — only to verify login by hashing the attempt and comparing. If the database leaks, hashed passwords resist reversal; encrypted passwords leak if the key is also stolen.

Question 16: What is bcrypt and what is a salt?

Answer: bcrypt is a password hashing algorithm designed to be slow (configurable cost factor). A salt is random data added to each password before hashing — two users with the same password get different hashes, defeating rainbow table attacks. bcrypt embeds the salt in the output. Use cost factor 10–12: `bcrypt.hash(password, 12)`. Never write your own hashing — use established libraries.

Question 17: What is multi-factor authentication (MFA)?

Answer: Multi-factor authentication requires two or more verification factors: something you know (password), something you have (phone authenticator app, SMS code), or something you are (fingerprint). Time-based One-Time Password (TOTP) apps like Google Authenticator generate 6-digit codes rotating every 30 seconds. MFA dramatically reduces account takeover even if passwords leak. Required at most product companies for employee accounts.

Question 18: What is Single Sign-On (SSO)?

Answer: Single Sign-On lets users log in once and access multiple applications without separate credentials. Enterprise SSO uses protocols like SAML or OIDC with an Identity Provider (IdP)

— e.g., Okta, Azure Active Directory. Employee logs into IdP, gets access to Jira, Confluence, and internal tools. Reduces password fatigue and centralizes access control and offboarding.

Question 19: What is Role-Based Access Control (RBAC)?

Answer: Role-Based Access Control assigns permissions to roles (admin, editor, viewer) instead of individual users. Users get one or more roles; roles define allowed actions. Example: `editor` can create posts; `viewer` can only read. Implement with middleware: `requireRole('admin')` checks `req.user.role`. Simpler than per-user permissions for most apps; Attribute-Based Access Control (ABAC) adds finer rules when needed.

Question 20: What is session-based authentication?

Answer: Session-based auth stores session data on the server (memory, Redis, database) with a session ID sent to the client as a cookie. On each request, the server looks up the session ID to identify the user. Pros: easy revocation (delete session), server controls lifetime. Cons: requires shared session store when scaling to multiple servers. Common in traditional server-rendered apps.

Question 21: When should you use sessions vs JSON Web Tokens (JWT)?

Answer: Sessions: server-rendered apps, when you need instant revocation, sensitive apps where token theft is a major concern. **JWT:** Single Page Applications, mobile apps, microservices, APIs consumed by third parties. **Hybrid (common):** short-lived JWT access token + refresh token in HTTP-only cookie, or session ID for web + JWT for mobile API. Match choice to client type and scale needs.

Question 22: What is Auth.js (NextAuth.js)?

Answer: Auth.js (formerly NextAuth.js) is a popular authentication library for Next.js and other frameworks. It supports OAuth providers (Google, GitHub), credentials login, email magic

links, and session management. Handles callbacks, CSRF protection, and cookie configuration. Reduces boilerplate for "Sign in with Google" and secure session handling. Widely used in Next.js MERN projects.

Question 23: What is Docker?

Answer: Docker is a platform that packages applications and their dependencies into containers — lightweight, isolated environments that run consistently on any machine. A container includes your Node.js app, runtime, libraries, and config. "Works on my machine" becomes "works everywhere." Docker uses images (blueprints) and containers (running instances). Essential for modern deployment at Amazon, Uber, and startups.

Question 24: What is the difference between a Docker image and a container?

Answer: An image is a read-only template — like a recipe or class. A container is a running instance of an image — like a dish or object. You build an image once (`docker build`), then create many containers from it (`docker run`). Images are layered and cached; containers are ephemeral (data lost when removed unless volumes are used). Push images to registries (Docker Hub, Amazon ECR) for deployment.

Question 25: What is a Dockerfile?

Answer: A Dockerfile is a text file with instructions to build a Docker image: base image (`FROM node:20-alpine`), copy files (`COPY . .`), install dependencies (`RUN npm ci`), expose port (`EXPOSE 3000`), and start command (`CMD ["node", "server.js"]`). Run `docker build -t myapp .` to create the image. Best practices: multi-stage builds, non-root user, small base images (Alpine), `.dockerignore` to exclude `node_modules`.

Question 26: What is a multi-stage Docker build?

Answer: Multi-stage builds use multiple `FROM` stages in one Dockerfile. Stage 1 (builder): install all dependencies, compile TypeScript, build Next.js. Stage 2 (production): copy only built artifacts and production dependencies — final image is much smaller without dev tools and source maps. Example: builder stage runs `npm run build`; final stage copies `.next/standalone` and runs `node server.js`.

Question 27: What is Docker Compose?

Answer: Docker Compose defines and runs multi-container applications with a `docker-compose.yml` file. Example: one service for Node.js API, one for MongoDB, one for Redis — all networked together. Commands: `docker compose up` starts all, `docker compose down` stops. Simplifies local development to match production architecture. Not for production orchestration at scale — use Kubernetes for that.

Question 28: What is a Docker volume?

Answer: Volumes persist data outside container lifecycle — when a container is deleted, volume data remains. Use for database files (MongoDB data directory), uploaded files, or logs. Define in Compose: `volumes: mongodb_data:/data/db`. Without volumes, database data is lost every time you recreate the container. Named volumes are managed by Docker; bind mounts map to host directories.

Question 29: What is Kubernetes (K8s) in simple terms?

Answer: Kubernetes is a container orchestration platform that manages running containers at scale across many machines. It handles: deploying new versions (rolling updates), restarting crashed containers, load balancing traffic, scaling replicas up/down, and secret management. Companies with hundreds of microservices use Kubernetes. For small apps, Docker on a single server or Platform-as-a-Service (Vercel, Railway) is simpler.

Question 30: What is Continuous Integration (CI)?

Answer: Continuous Integration is the practice of automatically building and testing code every time developers push changes. A CI pipeline runs on each pull request: install dependencies, lint, run unit tests, build the app. Catches bugs early before merge. Tools: GitHub Actions, GitLab CI, Jenkins, CircleCI. At product companies, no code merges without passing CI checks.

Question 31: What is Continuous Deployment (CD)?

Answer: Continuous Deployment automatically releases code to production after CI passes — sometimes multiple times per day. Continuous Delivery is similar but requires manual approval before deploy. A typical pipeline: push code → CI (test, build) → CD (deploy to staging → smoke tests → deploy to production). Reduces human error and speeds up shipping features. Flipkart and Uber deploy frequently with automated pipelines.

Question 32: What is GitHub Actions?

Answer: GitHub Actions is GitHub's built-in CI/CD platform. Define workflows in YAML files (`.github/workflows/ci.yml`) triggered by events (push, pull request). Jobs run on GitHub-hosted or self-hosted runners. Example workflow: on push to main → run tests → build Docker image → push to registry → deploy to cloud. Free tier for public repos; included minutes for private repos.

Question 33: What does a typical CI pipeline include for a MERN app?

Answer: (1) Checkout code. (2) Set up Node.js. (3) Install dependencies (`npm ci`). (4) Lint (`npm run lint`). (5) Run unit tests (`npm test`). (6) Build frontend (`npm run build`). (7) Optionally: integration tests with test database. (8) Build and push Docker image. (9) Deploy to staging/production. Fail fast — if lint fails, do not run tests. Cache `node_modules` for speed.

Question 34: What are environment secrets in CI/CD?

Answer: Secrets are sensitive values (API keys, database passwords, JSON Web Token secrets) stored securely in CI/CD platform settings — GitHub Secrets, GitLab Variables — never hardcoded in workflow files. Access in workflows: `${{ secrets.MONGODB_URI }}`. Rotate secrets periodically. Separate secrets per environment (staging vs production). Audit who can view and modify secrets.

Question 35: What is Infrastructure as Code (IaC)?

Answer: Infrastructure as Code defines cloud resources (servers, databases, networks) in version-controlled files instead of clicking in a cloud console. Tools: Terraform, AWS CloudFormation, Pulumi. Benefits: reproducible environments, code review for infrastructure changes, disaster recovery. Example: Terraform file defines an AWS Elastic Compute Cloud (EC2) instance and Amazon Relational Database Service (RDS) — `terraform apply` creates them.

Question 36: What is the difference between staging and production?

Answer: **Production** is the live environment real users access — highest stability and security requirements. **Staging** mirrors production closely for final testing before release — same config, separate database with test data. **Development** is local machines. Flow: develop locally → CI tests → deploy to staging → QA approval → deploy to production. Never test experimental features directly in production.

Question 37: What is a blue-green deployment?

Answer: Blue-green deployment runs two identical production environments: Blue (current live) and Green (new version). Deploy new code to Green, test it, then switch traffic from Blue to Green instantly. If problems occur, switch back to Blue — fast rollback. Requires double infrastructure temporarily. Alternative: rolling deployment updates instances gradually; canary deployment sends small traffic percentage first.

Question 38: What is monitoring and why does it matter?

Answer: Monitoring tracks application health in production: response times, error rates, CPU/memory usage, database connections. Tools: Datadog, New Relic, Prometheus + Grafana, AWS CloudWatch. Alerts notify on-call engineers when error rate spikes or latency exceeds thresholds. Without monitoring, you discover outages from angry users. Product companies expect you to understand basics of logs, metrics, and alerts.

Question 39: What is logging vs monitoring?

Answer: Logging records discrete events — "user 123 logged in", "payment failed for order 456" — useful for debugging specific issues. Monitoring aggregates metrics over time — requests per second, 99th percentile latency, error percentage — useful for trends and alerts. Use structured logging (JSON) sent to centralized log storage (ELK, CloudWatch Logs). Correlate logs with trace IDs across microservices.

Question 40: What is HTTPS and Transport Layer Security (TLS)?

Answer: HTTPS is HTTP encrypted with Transport Layer Security (TLS). It protects data in transit — passwords, tokens, personal info — from eavesdropping and tampering. Browsers show a padlock icon. Obtain certificates free from Let's Encrypt or via cloud providers. Redirect all HTTP to HTTPS. Set Strict-Transport-Security header. Required for production apps, especially with authentication cookies.

Question 41: What is a reverse proxy?

Answer: A reverse proxy sits in front of your application servers and handles incoming client requests — forwarding them to the appropriate backend. Nginx and AWS Application Load Balancer are common reverse proxies. Benefits: Transport Layer Security (TLS) termination, load balancing across multiple Node.js instances, caching static assets, rate limiting, and hiding internal server details. Clients see one entry point.

Question 42: What is a Content Delivery Network (CDN)?

Answer: A Content Delivery Network caches static assets (images, JavaScript, CSS, videos) on servers worldwide close to users. A user in Mumbai gets files from an Indian edge server instead of a United States origin — faster load times. Cloudflare, Amazon CloudFront, and Fastly are popular CDNs. Next.js on Vercel uses CDN automatically. Essential for global products like Netflix and Flipkart.

Question 43: What is `.env` file management across environments?

Answer: Store environment-specific config in `.env.development`, `.env.staging`, `.env.production` — never commit real secrets to git. Use `.env.example` with placeholder keys for documentation. In CI/CD, inject secrets from the platform's secret store. In Docker, pass via environment variables or secret mounts. Rotate credentials when team members leave. Tools like HashiCorp Vault manage secrets at enterprise scale.

Question 44: What is health check endpoint?

Answer: A health check endpoint (e.g., `GET /health`) returns 200 OK when the service is running and dependencies are reachable. Load balancers and Kubernetes probe it to know if instances are healthy — unhealthy instances are removed from traffic. Advanced checks verify database connectivity: `{ status: 'ok', db: 'connected' }`. Simple but critical for zero-downtime deployments and auto-recovery.

Question 45: What is the principle of least privilege?

Answer: Give users, services, and credentials only the minimum permissions needed — nothing more. Database user for the API should read/write app tables, not drop databases. Cloud Identity and Access Management (IAM) roles should allow specific actions on specific resources. If an API key leaks, limited scope reduces damage. Apply to human accounts, service accounts, and JSON Web Token claims.

Question 46: What is npm audit and dependency security?

Answer: `npm audit` scans dependencies for known vulnerabilities and suggests fixes. Run in CI pipelines — fail build on critical vulnerabilities. Keep dependencies updated, use lock files, prefer well-maintained packages. Supply chain attacks (malicious packages) are real — review new dependencies, use Dependabot for automated update pull requests. Companies like Google scan all third-party code before use.

Question 47: What is a Web Application Firewall (WAF)?

Answer: A Web Application Firewall filters HTTP traffic to block common attacks: SQL injection, Cross-Site Scripting, distributed denial-of-service. Cloud providers offer managed WAF (AWS WAF, Cloudflare). Rules block suspicious patterns, rate limit by Internet Protocol, and geo-block if needed. Not a substitute for secure code but adds a defense layer. Important for public-facing APIs at scale.

Question 48: How do Docker and CI/CD work together?

Answer: CI builds and tests code, then builds a Docker image tagged with the git commit hash. Push image to a container registry. CD pulls that image and deploys to servers or Kubernetes. Same image runs in dev (Compose), staging, and production — consistency guaranteed. Rollback = deploy previous image tag. This is the standard deployment flow at most modern product companies.

Question 49: What is zero-downtime deployment?

Answer: Zero-downtime deployment updates production without users noticing interruption. Techniques: rolling updates (replace instances one by one), blue-green switch, or Kubernetes rolling deployment with readiness probes. Requirements: health checks, graceful shutdown (finish in-flight requests before stopping), database migrations compatible with old and new

code (expand-contract pattern). Users at Uber and Amazon never see "site down for maintenance."

Question 50: How would you explain securing and deploying your MERN app in an interview?

Answer: Structure your answer: **Auth:** bcrypt passwords, JWT with refresh tokens in HTTP-only cookies, auth middleware on protected routes, role checks for admin actions. **Security:** helmet, HTTPS, input validation, rate limiting on login, CORS whitelist. **DevOps:** Dockerfile with multi-stage build, GitHub Actions CI (lint, test, build), deploy Docker image to cloud, environment secrets, health check endpoint, MongoDB Atlas for managed database. Mention one tradeoff you considered — e.g., JWT vs sessions for your use case.